Data Science & AI

DATA STRUCTURES Through Python

Queues

Lecture No.- 02

By- Satya sir

Recap of Previous Lecture

Queue

- Insertion & Deletion happens from Different ends
  (enqueue) (Dequeue)
- Types of Queues
  - Simple Queue } Insertion : rear
  - Circular Queue } Deletion : front — FIFO
  - Double Ended Queue : Insertion, Deletion } rear, front — LIFO | FIFO
  - Priority Queue : Insertion (rear), Deletion (front)
- Simple Queue

Topics to be Covered

- Circular Queue
- Deque
- Priority Queue
- Simple Queue Implementation in Python

Topic : Circular Queue

Empty : $f == -1$

Full : $f == (r+1) \% SIZE$

$r = -1, \boxed{f = -1}$

$f = -1, r = -1$

SIZE = 6

$\boxed{f = 0}, r = 0$

$\cancel{r = 0}, \cancel{f = 0}$ $\boxed{r = 0}$

$\boxed{f = 1}$

$\cancel{f = 1}$

$r = 1$

$\cancel{r = 2}$

$\cancel{f = 6}$

$\boxed{\cancel{r = 5}}$

2 $\cancel{f = 2}$

$\cancel{f = 2}$

$r = (r+1) \% SIZE$

$f = (f+1) \% SIZE$

$\Rightarrow$ if $f > r$ or $f = (r+1) \% SIZE$

$\boxed{f = -1}$

$r = -1$

Topic : Circular Queue

CQ[ ], N, f=-1, r=-1

```
def enqueue(CQ, value):
    if f==(r+1)%N:
        print('circular Queue is Full')
        return
    else:
        r=(r+1)%N
        CQ[r]=value
        if f==-1:        # first Insertion
            f=0
```

```
def Dequeue(CQ):
    if f==-1:
        Print('Circular Queue is Empty')
        return
    else:
        result = CQ[f]
        Print('Deleted Element is' result)
        f=(f+1)%N
        if f>r:  # Last Element Deleted
            f=-1
            r=-1
```

Topic : Double Ended QUEue (DEQUE)

Dequeue : operation

Deque (Deck) : Type of Queue

- In Deque, Insertion and deletion operations can be performed from both ends (rear, front)

- Deque can be developed in either of 2 ways:

1) input - restricted deque : Insertion : rear only, Deletion: rear, front

2) Output - restricted deque : Insertion : rear, front Deletion: front only

Topic : Double Ended QUEue (DEQUE)

Initially, $f=-1$, $r=-1$, MAX_SIZE $= N = 5$, $DQ[5]$

enqueue_rear(10) : $r=0$, $DQ[r]=10$, $f=0$

enqueue_rear(20) : $r=1$, $DQ[r]=20$

enqueue_rear(30) : $r=2$, $DQ[r]=30$

Dequeue_rear( ) : Delete 30 $r=r-1$, $r=1$

enqueue_front(40) : $f=f-1$, if $f=-1$: $f=N-1$, $DQ[f]=40$
$f=4$

enqueue_front(50) : $f=f-1$, $DQ[f]=50$

Dequeue_front( ) : Delete 50, $f=(f+1)\%N$

$r=(r+1)\%N$

$r=0$ $r=1$ $r=2$ $f=3$

$f=-1$
$r=-1$

10	20	30	50	40

$f=0$ $f=4$

	front	rear
enqueue	if $f=-1$: $f=0, r=0$ $f=f-1$ if $f=-1$ $f=N-1$ $f=f-1$	$r=(r+1)\%N$ if $f=-1$: $f=0$
Dequeue	$f=(f+1)\%N$	$r=r-1$ if $r=-1$: $r=N-1$

$r,f=-1$

$f=0$

10				

$r=0$

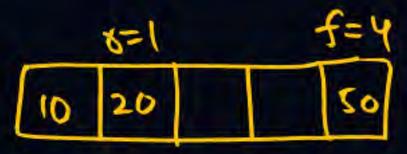
Topic : Double Ended QUEue (DEQUE)

Empty DQ

enqueue_rear() : $r=(r+1)\%N$, $f=0$

enqueue_front() : $f=0, r=0$

Non-Empty DQ



enqueue_rear() : $r=(r+1)\%N$

enqueue_front() : $f=f-1$

Dequeue_rear() : $r=r-1$

Dequeue_front() : $f=(f+1)\%N$

Topic : Priority Queue

$f = -1$
$r = -1$

0	1	2	3	4
10	30	40	30	20

~~f=0~~ r=0 r=1 r=2 f=3 f=4

enqueue_front (10) $f=0, r=0$

enqueue_front (20) $f = N-1, f=4$

enqueue_rear(30) $r = (r+1) \% N, \ r=1$

enqueue_rear (40) $r=2$

enqueue_front (50) $r=3$

Full: $f == (r+1) \% N$

Empty: $f == -1$

Deque as stack: when we Perform enqueue and dequeue operations from same end consecutively.

Otherwise: Deque will serve as Queue.

Simple

Topic : Queue Implementation

Simp

```
# implementing Queue using List :

q=[]
q.append(10)
q.append(100)
q.append(1000)
q.append(10000)
print("Initial Queue is:",q)
print(q.pop(0))
print(q.pop(0))
print(q.pop(0))
print("After Removing elements:",q)
```

```
def is_empty():
    return len(q) == 0

def size():
    return len(q)

print("Check queue is empty or not", is_empty())
print("Size of the queue:", size())
```

Topic : Queue Implementation

```
# implement queue using queue module

from queue import Queue
q=Queue(maxsize=4)
print("Initial Size Before Insertion:",q.qsize())
q.put('A')
q.put('AA')
q.put('AAA')
q.put('AAAA')
print("After Insertion:",q.qsize())
print("Queue is Full or Not:",q.full())
print("Size of Queue:",q.qsize())
```

```
print("Removing Elements:")
print(q.get())
print(q.get())
print(q.get())
print("Empty or Not??",q.empty())
print(q.get())
print("Empty or Not??",q.empty())
print("Size of Queue:",q.qsize())
```

Topic : Queue Implementation

Implementation Using deque module

```
from collections import deque
q=deque()
q.append(10)
q.append(100)
q.append(1000)
q.append(10000)
print("Initial Queue is:",q)
print(q.popleft())
print(q.popleft())
print("After Removing elements:",q)
```

2 mins Summary

- Circular Queue
- Double Ended Queue

To be Contd ...

THANK - YOU